

Graph Engineering & Decision Provenance

Moving AI Agents Beyond Ephemeral Chat Transcripts to Durable, Auditable State

Architectural Blueprint: Decision Provenance Agent

Stack: LangGraph • SQLite • ChromaDB • FastAPI

1 The Paradigm Shift: Evolution of AI Workflows

1. Vibe Coding

Human prompts with natural language, model writes single-shot code. Context is lost immediately after execution.

2. Agentic Engineering

Human orchestrates loops, tools, and evaluation gates. State lives ephemerally in running scripts or transcripts.

3. Graph Engineering

Agents read and write to **typed, queryable graphs** of lineage and evidence. Cross-session memory survives indefinitely.

The Core Insight: The bottleneck in production AI systems is rarely model intelligence. It is **where memory, lineage, and evaluation live**. A chat transcript is not a database.

2 The Dual-Graph Foundation: Lineage + Grounding

To eliminate hallucination and state amnesia, production systems connect two distinct graph planes:

LAYER	COMMIT DAG (LINEAGE)	KNOWLEDGE GRAPH (GROUNDING)
Core Question	"What changed, when, and by which run?"	"What exists, and what evidence supports it?"
Node / Edge Meaning	Nodes = Commits/Runs Edges = Parent mutations	Nodes = Entities/Claims Edges = Typed predicates (SUPPORTS, CITES)
Implementation	SQLite State Layer (Immutable event logs)	ChromaDB / Vector Graph (Semantic evidence chains)
Primary Action	Branch traversal & rollback	Evidence chain verification & conflict detection

3 Four Invariants for Every Graph Write

Our agent enforces four non-negotiable rules on every state modification:

1. Explicit Provenance

Every claim must cite a verified source edge or be explicitly tagged as an inference node. No ungrounded assertions.

2. Versioned Artifact Contracts

Every output is linked to its exact authoring agent ID, prompt hash, tool run ID, and semantic version.

3. Rubric-Bound Evaluations

Evaluators never output vague feedback ("looks good"). Every review identifies a deterministic rubric and missing edges.

4. Addressable Superseded State

Old decisions are **never deleted**. They are marked superseded, remaining queryable so you can audit *why decisions changed over time*.

4 Practical Architecture: Decision Provenance Agent

The 5-Plane Production Architecture

Control Plane

LangGraph workflow planner, budget & recursion guardrails.

Lineage Plane

SQLite commit history, state checkpoints, rollback pointers.

Evidence Plane

ChromaDB vector entities + grounded relation subgraphs.

Evaluator Grounding Contract (Structured Feedback):

```
{
  "decision": "revise",
  "target_claim": "Vendor X caused deployment latency spike in cluster A",
  "status": "UNSUPPORTED_PATH",
  "reason": "Missing relation edge between (Metric_Log_402) and (Vendor_X_API)",
  "required_evidence": ["Network trace logs", "Vendor SLA incident ticket"]
}
```

5 Production Audit Checklist

SYSTEM ATTRIBUTE	DESIGN QUESTION	STATUS
Reversibility	Can candidate decisions be rolled back without corrupting history?	Enforced via SQLite DAG
Entity Resolution	Are entity merges reversible to prevent catastrophic contamination?	Additive Aliases Kept
Grounding Layer	Does generation pull from bounded subgraphs instead of full transcript dumps?	1-2 Hop Subgraph Bounds

SYSTEM ATTRIBUTE	DESIGN QUESTION	STATUS
Decision Audit	Can a human query why decision A became decision B 10 days later?	Superseded Addressability

"The phrase the agent forgets, the graph does not." • Decision Provenance Agent Architecture Guide